

YI DONG

FPGA / RTL / Digital Design Intern

London, UK | 18125656904 | yd1723@ic.ac.uk | github.com/dydx404

Education

Imperial College London

London, UK

MEng Electronic and Information Engineering

Expected graduation: 2028

- Relevant coursework: Digital Electronics, Computer Architecture, Signals and Systems, Communications, Control Systems, Embedded Systems, Mathematics for Engineers, Software Systems.

Skills

FPGA / Digital Design: SystemVerilog, Verilog, FSMs, RTL datapaths, pipelined CPU basics, AXI4-Stream/Lite, AXI DMA usage, BRAM/FIFO basics, Vivado

Embedded / Systems: C/C++, Python, Bash, Linux, OpenCV, UDP sockets, PYNQ-Z1, Zynq-7000, PS/PL integration, ESP32

Verification / Tooling: Verilator, Icarus Verilog, Vivado/XSim, waveform debugging, directed tests, Python reference models, Git

Projects

FPGA Wand / PYNQ-Z1 Vision System

Feb 2026–Mar 2026

PYNQ-Z1, Zynq-7000, Verilog, AXI DMA, AXI-Stream, AXI-Lite/MMIO, OpenCV, UDP

GitHub

- Built a PYNQ-Z1 vision prototype integrating OpenCV camera capture, PS-side preprocessing, AXI DMA transfer, FPGA-side centroid extraction, and UDP point streaming.
- Implemented a simple AXI4-Stream RTL module for thresholded-frame centroid accumulation, exposing sum, count, frame_id, and valid status through AXI-Lite/MMIO.
- Integrated the Linux/Python runtime with PYNQ overlays, DMA buffer allocation, MMIO result readback, frame freshness checks, threshold tuning, and stroke packetization.
- Brought up the system on real PYNQ hardware; debugged camera initialization, DMA transfer behavior, overlay/HWH mismatches, duplicate frames, and noise-sensitive thresholds.

EE_Acceleration / Electromagnetic FDTD FPGA Accelerator

May 2026–Present

SystemVerilog, Vivado, XSim, Q3.13 fixed point, CORDIC, BRAM, 2D FDTD, renderer integration

GitHub

- Exploring FPGA implementation of an FDTD electromagnetic solver, focusing on fixed-point datapaths, BRAM-backed field storage, and SystemVerilog integration.
- Implemented Q3.13 RTL blocks for field update logic, CORDIC-based source generation, top-level control, and early solver/renderer interface wiring.
- Built a Python-assisted verification flow with Icarus Verilog, Verilator, Yosys, and Vivado/XSim; iterated a 64-cell 1D solver until RTL output matched the reference model.
- Ran Vivado implementation for the 1D top at 100 MHz (WNS 3.516 ns, 1169 LUTs, 1153 FFs, 2 DSPs); currently extending toward 2D solver and render buffer integration.

RV32I Pipelined CPU with Cache

Nov 2025–Dec 2025

SystemVerilog, RV32I pipeline, cache, hazards, forwarding, Verilator

GitHub

- Implemented and tested a simplified RV32I five-stage pipelined CPU in SystemVerilog as a teaching-oriented computer architecture project.
- Integrated forwarding paths, load-use hazard handling, stall/flush control, writeback selection, byte-addressable memory, and an experimental cache/refill path.
- Used Verilator/GoogleTest and directed RV32I assembly/reference programs to check load/store behavior, forwarding cases, pipeline control, and writeback results.
- Debugged PC flow, branch flushes, pipeline stalls, a0/writeback visibility, and cache interactions using VCD waveforms and text traces.

Smart Balcony Embedded Exploration

Personal exploration

ESP32, sensors, motors, GPIO/PWM

- Explored ESP32-based balcony automation prototypes using simple sensors and actuators, gaining hands-on practice with GPIO/PWM control, basic sensor acquisition, and embedded debugging.